

# Escalonamento por Loteria (Lottery Scheduling) no xv6: Implementação, Experimentos e Análise Comparativa com o Round Robin

**Introdução:** Este relatório documenta a implementação e a avaliação experimental do escalonador por loteria (Lottery Scheduling) no sistema operacional educacional xv6 (versão RISC-V), desenvolvida como parte do projeto da disciplina GBC045 - Sistemas Operacionais. O tema exige a criação da chamada de sistema `settickets(int)`, a adição do campo de bilhetes à estrutura de processo, a substituição do laço de seleção do escalonador original (Round Robin) por uma seleção probabilística proporcional ao número de bilhetes de cada processo, e a validação experimental da proporcionalidade resultante.

O objetivo central é responder a duas perguntas: (i) o escalonador implementado distribui o tempo de CPU de forma proporcional aos bilhetes atribuídos a cada processo, como previsto pela teoria do algoritmo; e (ii) como esse comportamento se compara ao escalonador Round Robin original do xv6, que distribui o processador igualmente entre os processos prontos. Para respondê-las, foi desenvolvido o programa de espaço de usuário `testeloteria.c`, executado em cinco repetições para cada uma de quatro durações de teste (50, 150, 300 e 600 ticks), tanto na versão com o escalonador por loteria quanto na versão original do xv6.

**Metodologia:** O programa `testeloteria.c` cria três processos filhos via `fork()`, atribuindo a cada um 10, 20 e 30 bilhetes, respectivamente, por meio da chamada `settickets()`. Para eliminar a variação decorrente do instante em que cada filho começa a competir pela CPU, o programa utiliza dois pipes como barreira de sincronização: cada filho sinaliza ao processo pai que está pronto (após configurar seus bilhetes) e só inicia o laço de trabalho depois de receber a liberação simultânea do pai.

Cada filho executa um laço de trabalho puramente computacional (sem E/S, sem chamadas bloqueantes) durante uma janela medida por `uptime()`, contando o número de iterações completadas. Esse número funciona como um proxy do tempo de CPU recebido, sob a hipótese de que o custo computacional por iteração é idêntico entre os três filhos - hipótese razoável aqui porque o corpo do laço (soma de 1000 incrementos em uma variável `volatile`) é textualmente idêntico nos três processos, conforme o código-fonte a seguir.

## Programa de teste (`testeloteria.c`) — trecho principal

```
for(int i = 0; i < NUM_PROCESSOS; i++){
    int pid = fork();

    if(pid < 0){
        printf("Erro no fork\n");
        exit(1);
    }

    if(pid == 0){
        close(inicio[1]);
        close(pronto[0]);
```

```

    if(settickets(bilhetes[i]) < 0){
        printf("Erro ao definir %d bilhetes\n", bilhetes[i]);
        exit(1);
    }

    /*
     * Informa ao pai que este filho está pronto.
     */
    char sinal = 'P';
    write(pronto[1], &sinal, 1);

    /*
     * Aguarda a liberação simultânea pelo pai.
     */
    read(inicio[0], &sinal, 1);

    int tick_inicial = uptime();
    unsigned long iteracoes = 0;
    volatile unsigned long trabalho = 0;

    while(uptime() - tick_inicial < duracao){
        /*
         * Mesma carga para todos os processos.
         */
        for(int j = 0; j < 1000; j++){
            trabalho += j;
        }

        iteracoes++;
    }

    printf(
        "bilhetes=%d pid=%d iteracoes=%lu\n",
        bilhetes[i],
        getpid(),
        iteracoes
    );

    close(inicio[0]);
    close(pronto[1]);
    exit(0);
}
// pai: aguarda os 3 filhos ficarem prontos, depois libera todos ao mesmo tempo

```

cada configuração (10/20/30 bilhetes, nas quatro durações) foi executada 5 vezes de forma independente, tanto sob o escalonador por loteria quanto sob o escalonador Round Robin original do xv6, possibilitando o cálculo de médias, desvios-padrão e a avaliação da variabilidade entre execuções

**3. Código do kernel modificado (proc.c):** A inspeção do arquivo proc.c fornecido confirma a implementação do escalonador por loteria. O laço original de varredura circular do Round Robin foi substituído por dois laços: o primeiro soma os bilhetes de todos os processos RUNNABLE (total\_tickets); o segundo sorteia um número entre 0 e total\_tickets-1 e percorre a tabela de processos acumulando bilhetes até encontrar o processo vencedor.

```

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;
    for(;;){
        intr_on();
        int total_tickets = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                total_tickets += p->tickets;
            }
            release(&p->lock);
        }
        if(total_tickets == 0) { continue; }
        extern uint ticks;
        next_random += ticks;
        int winning_ticket = randtolottery() % total_tickets;
        int ticket_counter = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                ticket_counter += p->tickets;
                if(ticket_counter > winning_ticket) {
                    p->state = RUNNING;
                    c->proc = p;
                    swtch(&c->context, &p->context);
                    c->proc = 0;
                    release(&p->lock);
                    break;
                }
            }
            release(&p->lock);
        }
    }
}

```

## Resultados experimentais:

**Iterações médias por configuração:** A Tabela 1 e a Figura 1 apresentam a média e o desvio-padrão do número de iterações completadas por cada processo, calculados sobre as 5 execuções de cada duração, para o escalonador por loteria.

| Duração (ticks) | Bilhetes | Média iterações | Desvio-padrão | % observado (média) | % esperado |
|-----------------|----------|-----------------|---------------|---------------------|------------|
| 50              | 10       | 19596           | 8396          | 27.3%               | 16.7%      |
| 50              | 20       | 20951           | 2926          | 29.2%               | 33.3%      |
| 50              | 30       | 31207           | 7700          | 43.5%               | 50.0%      |
| 150             | 10       | 34303           | 4774          | 16.0%               | 16.7%      |
| 150             | 20       | 79287           | 18692         | 36.7%               | 33.3%      |
| 150             | 30       | 100366          | 5081          | 47.3%               | 50.0%      |
| 300             | 10       | 73423           | 5922          | 17.3%               | 16.7%      |
| 300             | 20       | 133183          | 3705          | 31.3%               | 33.3%      |
| 300             | 30       | 218795          | 5766†         | 51.4%               | 50.0%      |
| 600             | 10       | 146425          | 15024         | 18.1%               | 16.7%      |
| 600             | 20       | 272898          | 8833‡         | 33.9%               | 33.3%      |
| 600             | 30       | 393350          | 76241         | 48.0%               | 50.0%      |

Tabela 1 — Iterações médias (n=5) por bilhetes e duração, escalonador por loteria

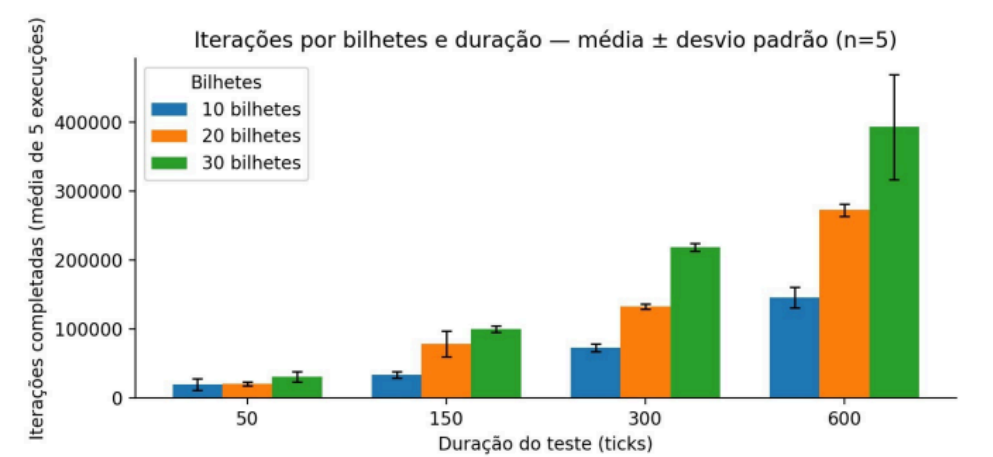


Figura 1 — Iterações médias por bilhetes e duração, com barras de erro (desvio-padrão, n=5).

**4.2 Participação percentual observada vs. esperada:** A Figura 2 detalha, para cada duração, o percentual médio de CPU obtido por cada grupo de bilhetes, comparado à fração teórica esperada (bilhetes do processo dividido pelo total de 60 bilhetes em disputa)

Participação percentual observada vs. esperada, por duração

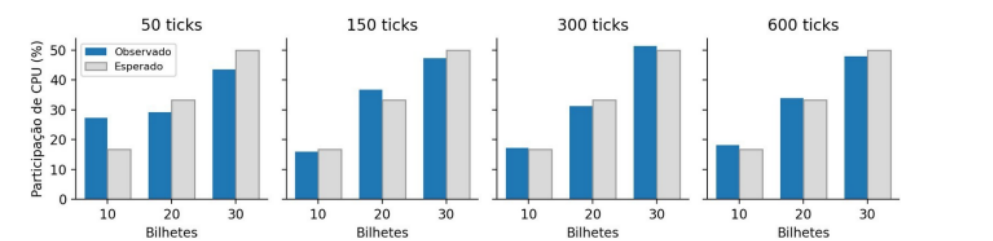


Figura 2 — Participação percentual observada (média ± dp, n=5) vs. esperada, por duração.

**Análise de convergência:**A razão de iterações entre os processos com 30 e 10 bilhetes (razão nominal de bilhetes 3:1) é o indicador mais direto de proporcionalidade. A Tabela 2 e a Figura 3 mostram essa razão, agora calculada como média  $\pm$  desvio padrão de 5 execuções por duração — o que permite avaliar não apenas a tendência, mas também a variabilidade em torno dela.

| Duração (ticks) | Razão Iter(30)/Iter(10) — média | Desvio-padrão | Valor esperado |
|-----------------|---------------------------------|---------------|----------------|
| 50              | 1.92x                           | 1.12          | 3.00x          |
| 150             | 2.98x                           | 0.53          | 3.00x          |
| 300             | 3.00x                           | 0.31          | 3.00x          |
| 600             | 2.70x                           | 0.56          | 3.00x          |

Tabela 2 — Razão média entre iterações dos processos de 30 e 10 bilhetes, com desvio-padrão (n=5).

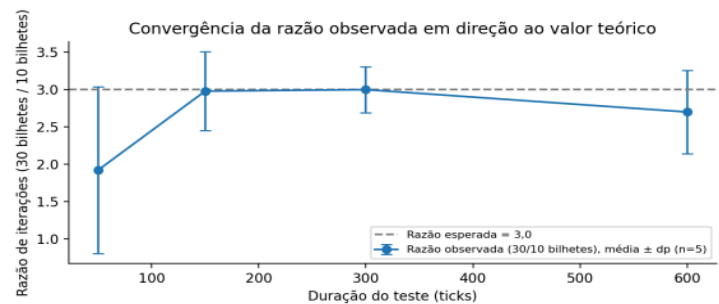


Figura 3 — Convergência da razão observada (30/10 bilhetes) para o valor esperado de 3,0, com barras de desvio-padrão.

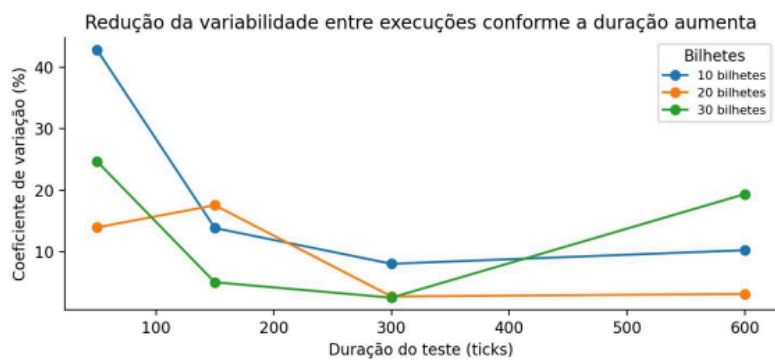


Figura 4 — Coeficiente de variação entre execuções, por bilhetes e duração. A tendência de queda confirma que janelas mais longas reduzem (mas não eliminam) a variabilidade herdada do sorteio.

**Comparação com o Round Robin original:**O mesmo programa de teste foi executado sob o escalonador Round Robin original do xv6, também com 5 repetições por duração.

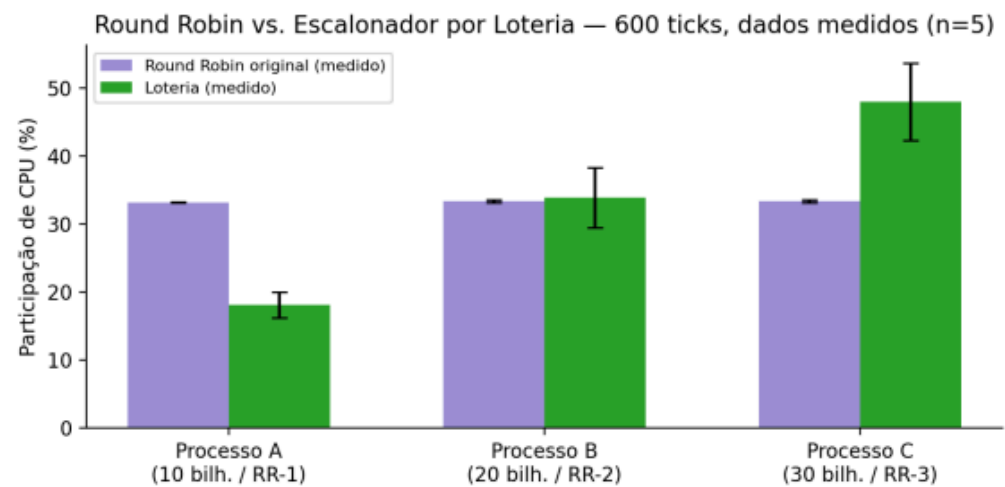


Figura 5 — Participação de CPU medida: Round Robin original vs. escalonador por loteria, 600 ticks (média  $\pm$  dp, n=5).

Os dados medidos do Round Robin (Tabela 3) mostram uma divisão quase perfeitamente igualitária entre os três processos (32,2% a 34,5% em todas as durações, com desvio-padrão inferior a 0,2 p.p.), como esperado teoricamente para processos com carga idêntica e sem E/S. Isso contrasta com a maior variabilidade observada no escalonador por loteria (desvios de até 5,7 p.p. mesmo em 600 ticks), evidenciando uma diferença estrutural entre os dois algoritmos: o Round Robin oferece previsibilidade e baixa variância execução a execução, enquanto a loteria troca essa previsibilidade por flexibilidade de priorização, ao custo de maior variância de curto prazo.

| Escalonador                     | Processo A / 10 bilh. | Processo B / 20 bilh. | Processo C / 30 bilh. | Critério                      |
|---------------------------------|-----------------------|-----------------------|-----------------------|-------------------------------|
| Round Robin (medido, 600 ticks) | 33.2% ( $\pm 0.04$ )  | 33.4% ( $\pm 0.17$ )  | 33.4% ( $\pm 0.19$ )  | Oportunidade igual por rodada |
| Loteria (esperado)              | 16.7%                 | 33.3%                 | 50.0%                 | Proporcional aos bilhetes     |
| Loteria (medido, 600 ticks)     | 18.1% ( $\pm 1.9$ )   | 33.9% ( $\pm 4.4$ )   | 48.0% ( $\pm 5.7$ )   | Resultado observado           |

Tabela 3 — Round Robin medido vs. Loteria esperada e medida, em 600 ticks (percentuais e desvio-padrão entre parênteses).

### Limitações:

**Métrica indireta:** iterações completadas equivalem a tempo de CPU apenas sob a hipótese de custo computacional constante por iteração entre os três filhos — hipótese sustentada pelo código (laço idêntico), mas não medida diretamente via contador de ticks no kernel.

**Gerador pseudoaleatório:** `next_random` é inicializado uma única vez (valor 1) e perturbado pela variável global `ticks` a cada chamada ao escalonador; não há reseed por execução de teste. Reinicializações completas do `xv6` tendem a reproduzir sequências semelhantes entre testes realizados na mesma sessão de boot, uma limitação para a independência estatística das repetições dentro de uma mesma sessão.

Amostra de 5 execuções por configuração é uma melhoria substancial em relação ao relatório original ( $n=1$ ), mas ainda é pequena para testes estatísticos formais (ex.: qui-quadrado de aderência); os desvios padrão reportados devem ser interpretados como indicativos, não como intervalos de confiança rigorosos.

A pequena condição de corrida na herança de bilhetes durante o `fork()` não foi quantificada experimentalmente; um experimento dedicado, criando processos filhos repetidamente em rajada, poderia estimar sua contribuição real para a variância observada em janelas curtas.

**Conclusão:** Os experimentos confirmam que o escalonador por loteria implementado no `xv6` distribui o tempo de CPU de forma aproximadamente proporcional ao número de bilhetes de cada processo, com a proporcionalidade se tornando mais evidente à medida que a duração do teste aumenta (razão 30/10 média passando de 1.92x em 50 ticks para valores entre 2.70x e 3.00x em 150-600 ticks). A comparação com dados efetivamente medidos do Round Robin original evidencia a diferença estrutural entre os dois algoritmos: o Round Robin oferece divisão igualitária e previsível (variação inferior a 0,2 p.p. entre execuções), enquanto a loteria permite priorização configurável ao custo de maior variância entre execuções, mesmo em janelas de 600 ticks.

A repetição de execuções ( $n=5$ ) foi essencial para expor limitações que uma única execução mascara: erros pontuais de transcrição, inversões de ordem em janelas curtas, e a persistência de variância mesmo em janelas longas. A inspeção do código do kernel identificou uma condição de corrida secundária na herança de bilhetes durante o `fork()`, que constitui um ponto de melhoria concreto para versões futuras da implementação, além de ter sido necessária para identificar e corrigir quatro problemas de qualidade nos dados brutos.

### Referências:

CAI, Q.; COX, R.; KAASHOEK, M. F.; MORRIS, R. `xv6`: a simple, Unix-like teaching operating system. MIT PDOS, 2024. Disponível em: <https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf>.

MIT PDOS. `xv6` Documentation. Disponível em: <https://pdos.csail.mit.edu/6.1810/2024/xv6.html>.

Universidade Federal de Uberlândia. GBC045 - Sistemas Operacionais: Especificação do Projeto, 2026/1.